Nombre: Noemi Mata Ruiz Fecha: 15/10/2026

Conceptos de Estructuras Dinámicas en Java

1. Definición de Estructuras

Una estructura de datos **dinámica** es aquella cuyo tamaño puede crecer o decrecer durante la ejecución del programa. A diferencia de las estructuras **estáticas**, la memoria se solicita y se libera conforme se necesita.

En Java, cuando queremos crear un nuevo objeto o nodo en tiempo de ejecución, utilizamos la palabra reservada **new**. Este espacio de memoria se asigna en una región llamada **heap**. Las conexiones entre los elementos no dependen de su posición física, sino de las **referencias (punteros)** que guardan la dirección del siguiente elemento. Cuando un nodo ya no es accesible, el **Garbage Collector** se encarga de liberar esa memoria automáticamente.

> **Nota Importante**
> El acceso en una lista dinámica es **secuencial**. Para llegar al elemento n, debemos recorrer todos los anteriores, lo que resulta en una complejidad de *O(n)*.

2. Características Principales

Completa la siguiente lista de características clave de las estructuras dinámicas basándote en la lectura:

- **Tamaño variable:** Se ajusta a la cantidad real de datos en uso.
- **Memoria no contigua:** Los elementos pueden estar dispersos y se conectan mediante enlaces.
- **Eficiencia:** Solo se reserva el espacio estrictamente necesario.
- **Flexibilidad:** La inserción y eliminación no requieren desplazar otros elementos.
- **Costo de memoria:** Cada nodo requiere espacio extra para almacenar sus enlaces.

3. Comparación: Arreglo vs. Lista Enlazada

Analiza las diferencias técnicas entre ambas estructuras y completa los espacios vacíos en la tabla.

Aspecto	Arreglo (Estático)	Lista Enlazada (Dinámica)
Tamaño	Fijo (definido al inicio)	Variable: crece o se achica según se necesite
Acceso por índice	O(1) (Directo)	O(n): hay que ir pasando nodo por nodo
Insertar al inicio	O(n) (Requiere desplazar)	O(1): solo se cambian unos enlaces
Uso de Memoria	Bloque contiguo	Nodos dispersos, unidos por enlaces

4. Análisis de Implementación

¿Por qué es crítico el orden al reasignar referencias durante una inserción? Explica qué sucedería si desconectas el nodo anterior antes de conectar el nuevo nodo a la lista.

Si sueltas primero el enlace anterior, ya no hay forma de llegar al resto de la lista.

Es como soltar una cuerda antes de agarrar la otra: lo que estaba abajo se cae.

Esos nodos se quedan sin nadie que los señale y Java los borra como basura.

Por eso primero se conecta el nodo nuevo y al final se cambia el enlace anterior.

---

5. Si una estructura requiere recorrer todos los elementos anteriores para encontrar el dato buscado, ¿cuál es su complejidad temporal?

A) O(log n)
B) O(1)
C) O(n)

Nombre: Noemi Mata Ruiz Fecha: 15/10/2026

Diagramación de Referencias y Nodos

Regla de Oro

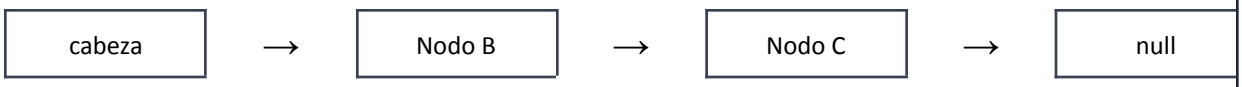
Para no perder el acceso al resto de la lista, **primero conecta el nuevo nodo** a la secuencia existente y **después actualiza la referencia previa**. Si desconectas antes de asegurar el enlace, el Garbage Collector podría eliminar los nodos huérfanos.

En una lista enlazada simple, cada nodo tiene un valor y una referencia al siguiente nodo (next). A continuación, realizarás el trazado visual de cómo cambian estas referencias durante operaciones comunes.

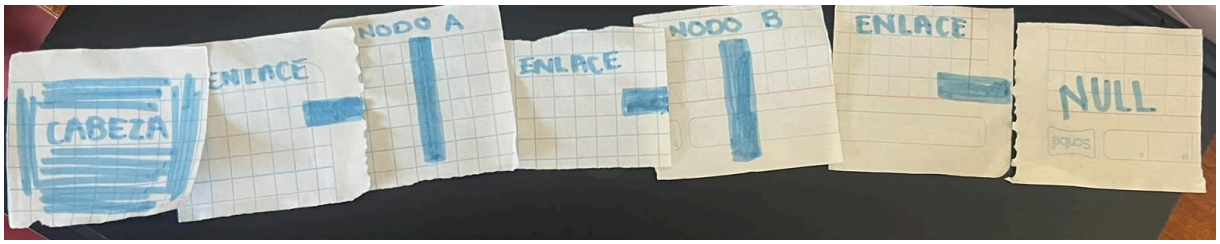
Escenario 1: Insertar al inicio

Se tiene una lista con los valores [B] -> [C]. Se desea insertar un nuevo nodo [A] al principio de la lista. Dibuja cómo deben quedar las flechas de referencia.

Estado inicial (antes)



Tu diagrama (después)



Primero el nodo A apunta al nodo B, y la cabeza ahora apunta a A y como resultado da: **cabeza → Nodo A → Nodo B → Nodo C → null**.

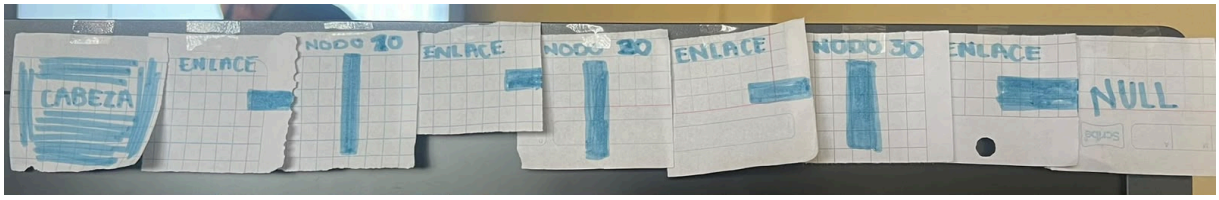
Escenario 2: Insertar entre dos nodos

Se tiene la lista [10] -> [30]. Se desea insertar el nodo [20] entre el 10 y el 30. Muestra el orden correcto de las conexiones para no perder el nodo 30.

Estado inicial (antes)

Nodo 10 → Nodo 30 → null

Tu diagrama (después)



cabeza →Nodo 10 → Nodo 20 → Nodo 30 → null

Primero el nodo 20 apunta al 30 y después el nodo 10 apunta al nodo 20.

Escenario 3: Eliminar el último nodo

Se tiene la lista [X] -> [Y] -> [Z]. Se desea eliminar el nodo [Z]. Dibuja qué referencia debe cambiar a null y qué sucede con el puntero del nodo penúltimo.

Estado inicial (antes)

Nodo X → Nodo Y → Nodo Z → null

Tu diagrama (después)



cabeza →Nodo X → Nodo Y → null (Nodo Z queda suelto y Java lo elimina)

Primero en enlace de Y apunta a null y Z se queda sin enlace y Java lo borra.

Pregunta de Reflexión:

¿Por qué en el Escenario 2 es vital conectar [20].next = [30] antes de hacer [10].next = [20]?

Porque el 30 solo está enlazado desde el 10. Si haces [10].next = [20] primero,

el 10 deja de apuntar al 30 y ya nadie sabe dónde quedó el 30 (se pierde).

Al hacer antes [20].next = [30], el nodo nuevo ya conoce al que sigue,

y después se puede acomodar el 10 sin perder nada.

Nombre: Noemi Mata Ruiz Fecha: 15/10/2026

Laboratorio de Programación: Implementación de una Lista Enlazada

En esta actividad, aplicarás tus conocimientos sobre estructuras dinámicas para construir una lista enlazada simple en Java. Deberás completar el código siguiendo los comentarios TODO. Recuerda que en una estructura dinámica, la memoria se gestiona mediante referencias en el *heap*.

Recordatorio de Lógica

1. **Nodo**: Es el bloque básico que contiene el dato y el enlace al siguiente.
2. **Referencia**: En Java, usamos variables de objeto para apuntar a la dirección de memoria del siguiente nodo.
3. **Orden de Inserción**: Para no perder la lista, primero conecta tu nuevo nodo a la cabeza actual y luego actualiza la cabeza.

Parte 1: La Clase Nodo

Completa el constructor para inicializar el nodo con un valor entero.

```
public class Nodo {
    int dato;
    Nodo siguiente;

    // TODO: Implementar el constructor
    public Nodo(int dato) {
        this.dato = dato;
        this.siguiente = null;
    }
}
```

Parte 2: La Clase ListaSimple

Implementa los métodos para gestionar la estructura. Presta especial atención al método insertarAlInicio para asegurar que la complejidad sea O(1).

```
public class ListaSimple {
    private Nodo cabeza;

    public ListaSimple() {
        this.cabeza = null;
    }

    // TODO: Implementar insertarAlInicio
```

```
    public void insertarAlInicio(int valor) {
        Nodo nuevo = new Nodo(valor);
        nuevo.siguiente = cabeza;
        cabeza = nuevo;
    }

    // TODO: Implementar recorrer (imprimir todos los elementos)
    public void recorrer() {
        Nodo actual = cabeza;
        while (actual != null) {
            System.out.println(actual.dato);
            actual = actual.siguiente;
        }
    }
}
```

Parte 3: Análisis de Casos Límite

Una vez completado el código, analiza cómo se comportaría tu implementación en las siguientes situaciones.

Caso Límite	¿Qué sucede en tu código?
La lista está vacía (cabeza == null)	El nuevo nodo queda como el único y apunta a null. Al recorrer, el ciclo no empieza, no imprime nada y no hay error.
La lista tiene un solo elemento	La cabeza apunta a ese nodo y su siguiente es null. Al recorrer imprime ese dato y termina.
Insertar un nuevo nodo cuando ya existen otros	El nuevo nodo se engancha al que era el primero y luego pasa a ser la cabeza. Nadie se pierde.

Pregunta de Reflexión

¿Por qué es necesario usar una variable temporal (como actual) en el método recorrer en lugar de mover directamente la variable cabeza?

Porque si mueves la cabeza pierdes el inicio de la lista y ya no puedes volver al primer nodo.

La variable actual es como un dedo que va recorriendo la lista, mientras la cabeza se queda

quieta marcando dónde empieza. Así la lista no se daña al recorrerla.

Nombre: Noemi Mata Ruiz Fecha: 15/10/2026

Evaluación: Estructuras Dinámicas en Java

Parte 1: Análisis de Complejidad (Big O)

Compara el rendimiento de los arreglos estáticos frente a las listas enlazadas dinámicas seleccionando la respuesta correcta.

1. ¿Cuál es la complejidad temporal para acceder a un elemento por su índice en una **lista enlazada**?

A) O(1)
B) O(n)
C) O(log n)

2. Al **insertar un elemento al inicio**, ¿qué estructura es más eficiente?

A) El arreglo, porque tiene acceso O(1).
B) La lista enlazada, porque solo requiere reasignar referencias O(1).
C) Ambas tienen la misma eficiencia O(n).

3. ¿Por qué un **arreglo** tiene una complejidad de O(1) para el acceso por índice?

A) Porque utiliza memoria no contigua.
B) Porque el Garbage Collector optimiza la búsqueda.
C) Porque al estar en memoria contigua, se puede calcular la dirección exacta del dato.

4. Si necesitas buscar un dato específico recorriendo todos los elementos anteriores, la complejidad es:

A) O(n)
B) O(1)
C) $O(n^2)$

5. ¿Cuál es una desventaja de la lista enlazada en términos de eficiencia de memoria?

A) La memoria es estática y no se puede liberar.
B) Ocupa menos espacio que un arreglo.
C) Cada nodo consume memoria extra para almacenar la referencia al siguiente elemento.

Parte 2: Gestión de Memoria en Java

Responde Verdadero (V) o Falso (F) a las siguientes afirmaciones sobre el manejo de memoria.

6. En Java, los objetos creados con la palabra new se almacenan en una región de memoria llamada *heap*.

A) Verdadero

B) Falso

7. El *Garbage Collector* libera automáticamente la memoria de los nodos que aún tienen una referencia activa apuntando a ellos.

A) Falso

B) Verdadero

8. Una estructura dinámica se diferencia de una estática porque su tamaño puede crecer o decrecer durante la ejecución.

A) Falso

B) Verdadero

Parte 3: Lógica de Referencias y Fugas de Memoria

Responde de manera detallada a las siguientes preguntas sobre la manipulación de nodos.

9. Explica por qué es crítico conectar el nuevo nodo a la lista **antes** de actualizar la referencia de la 'cabeza' durante una inserción al inicio.

Porque la cabeza es lo único que nos dice dónde empieza la lista. Si la cambias

primero, el nodo nuevo todavía no apunta a nada y la lista vieja se pierde.

Primero se hace nuevo.siguiente = cabeza (el nuevo ya conoce a la lista) y después

cabeza = nuevo. Así el nodo nuevo queda al frente y nadie se pierde.

---

10. ¿Qué sucede con la memoria en Java si desconectas accidentalmente el primer nodo de una lista sin haber guardado una referencia al resto de los elementos? Justifica tu respuesta mencionando el concepto de *Garbage Collector*.

Esos nodos quedan huérfanos: ya nadie los señala y no hay forma de volver a ellos.

Entonces el Garbage Collector, que es el recolector de basura de Java, los detecta

y libera esa memoria automáticamente. Los datos se pierden, pero no hay fuga de

memoria porque Java la limpia solo (en otros lenguajes sí sería un problema).
